Sim-to-Real Transfer in Robotic Control via Reinforcement Learning and Domain Randomization

Mattiahs Bubich s347510 | Stefano Cardella s341749 | Lorenzo Ferrandi s349309 | Parastoo Tavakoli Basti s336976

***Abstract*—In this paper, we explore the application of Reinforcement Learning (RL) techniques for robotic control in simulation, focusing on the sim-to-real transfer problem. The primary objective is to design and train control policies in a simulated environment using policy gradient algorithms. Two basic policy gradient algorithms are implemented – REINFORCE and Actor-Critic – from scratch, followed by the integration of an advanced RL algorithm, Proximal Policy Optimization (PPO), using a third-party library. To simulate the reality gap, two Hopper environments are employed, differing in some physical parameters. The project investigates how training in a randomized source domain can yield policies capable of maintaining performance in a shifted target domain, using Uniform Domain Randomization (UDR) of link masses. In addition, the project includes a personal extension proposed by the Teaching Assistants. The full implementation is available at: https://github.com/stefanocardella01/ReinforcementLearning.git**

I. Introduction

The goal of this project is to study reinforcement learning (RL) algorithms in the context of robotic control and policy transfer across simulated domains. The study is conducted using the `Hopper` environment from OpenAI Gym, a classic continuous control benchmark where a one-legged robot learns to move forward through trial and error.

The investigation begins by with the implementation and evaluation of policy gradient methods, starting from REINFORCE and progressing to Actor-Critic algorithms. Two Actor-Critic variants are considered: one based on temporal-difference learning (TD(0)), and another using Monte Carlo returns. These methods are compared in terms of sample efficiency, training stability, and final performance.

Subsequently, Proximal Policy Optimization (PPO), a more advanced on-policy algorithm,is introduced to define upper and lower performance bounds for policy transfer. A sim-to-real scenario is simulated by training and testing policies on distinct domains (source and target), where the agent's torso mass is modified to induce a dynamics mismatch.

To enhance generalization and robustness, Uniform Domain Randomization (UDR) is implemented by training the policy over a distribution of dynamics parameters. The analysis is further extended to the `Reacher` environment, testing the scalability of the learned methods across different robotic tasks.

II. Gym Hopper Environment

The Hopper environment in OpenAI Gym simulates a planar one-legged robot learning to hop forward without falling. It is implemented using the MuJoCo physics engine and serves as a standard benchmark for testing reinforcement learning algorithms in continuous control settings.

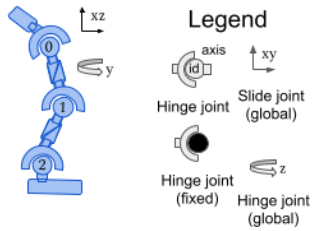


Fig. 1. Hopper legend.

A. Robot Structure

The robot Figure 1 is composed of four rigid segments: `torso`, `thigh`, `leg` and `foot`.

These are connected by three hinge joints: `thigh_joint`, `leg_joint`, and `foot_joint`, which correspond to the hip, knee, and ankle, respectively.

B. Action Space

The action space is a continuous 3D vector of joint torques. Each value controls a joint and lies within the interval $[-1.0, 1.0]$. The action vector is: `[torque_hip, torque_knee, torque_ankle]`

C. Observation Space

The state space is an 11-dimensional continuous vector. It includes:

- **5 position components**: torso height and angle, and joint angles
- **6 velocity components**: linear and angular velocities of torso and joints

The observation does not include the absolute horizontal position, making the task translation-invariant.

III. REINFORCE

The REINFORCE algorithm is a fundamental Monte Carlo policy gradient method used in reinforcement learning to directly optimize a stochastic policy with respect to the expected return. Unlike value-based methods, REINFORCE updates the policy parameters θ via stochastic gradient ascent on the expected performance objective:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] \tag{1}$$

where τ denotes a trajectory sampled from the policy π_θ, and $R(\tau)$ is the total return accumulated along that trajectory.

At each episode, the agent interacts with the environment by observing the current state, sampling an action from the policy $\pi_\theta(a|s)$, receiving a reward, and repeating this process until a terminal state is reached. During this interaction, the agent records the log-probabilities of the selected actions and their corresponding rewards.

After the episode concludes, the policy is updated using the following gradient estimate:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t] \tag{2}$$

where G_t denotes the return from time step t to the end of the episode. This update rule encourages the agent to increase the probability of actions that resulted in higher returns, thereby improving the expected performance of the policy. The gradient is then used to adjust the policy parameters through backpropagation.

REINFORCE is model-free, simple to implement, and produces unbiased gradient estimates. However, because it relies on full-episode returns and does not incorporate a value baseline, Figure 2 training exhibits high variability, with average returns fluctuating between 100 and 400 throughout the entire learning process. While the agent occasionally achieves high rewards, convergence remains unstable, with frequent oscillations even in the later stages of training. As a result, learning progresses slowly and inefficiently.

To mitigate this issue, a baseline function $b(s_t)$ is often introduced to reduce variance without introducing bias. The gradient estimate is modified accordingly:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot (G_t - b(s_t))] \tag{3}$$

This modification reduces the variance of the gradient estimates by normalizing the returns with respect to a baseline value, thus leading to more stable and efficient learning.

To address this, several constant baseline values (20, 80, 140, and 200) were tested to assess their effect on training stability and convergence speed, as shown in Figure 3. The figure shows that introducing a baseline yields faster and more stable convergence. However, setting the baseline too high (e.g., above 200) introduces an higher variance, even though it can reach higher rewards.

These results highlight the importance of selecting an appropriate baseline. A well-calibrated baseline can significantly improve learning stability and accelerate convergence. In contrast, a poorly chosen baseline—especially one that is too high—can undermine learning and reduce the overall effectiveness of the algorithm.

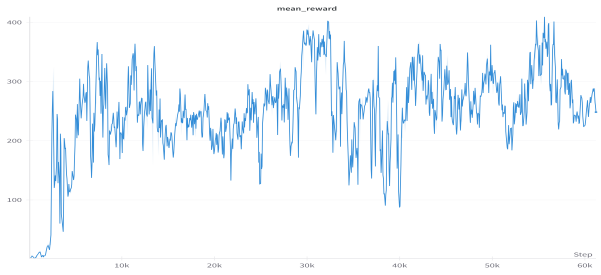


Fig. 2. REINFORCE without baseline.

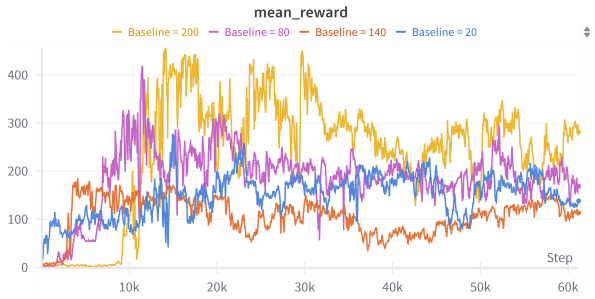


Fig. 3. REINFORCE with constant baselines (20, 80, 140, 200)..

IV. Actor-Critic

Actor-Critic algorithms combine the advantages of policy gradient methods and value function approximation by maintaining two distinct structures: an *actor*, which adjusts the policy parameters in the direction suggested by the critic, and a *critic*, which evaluates the action taken by the actor using a value function.

Let $\pi(a|s;\theta)$ denote a stochastic policy parameterized by θ, and let $\hat{v}(s;a)$ be the state-value function parameterized by w. The objective is to maximize the expected return.

According to the policy gradient theorem, the gradient of the objective function can be expressed as:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi(A_t|S_t;\theta) \cdot A^\pi(S_t, A_t)] \tag{4}$$

Here, $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ is the advantage function. In practice, this quantity must be estimated. Two common strategies are implemented to approximate the advantage: temporal-difference (TD) error and Monte Carlo return.

A. TD(0) Actor-Critic

The TD(0) variant of the Actor-Critic algorithm updates the value function using the one-step temporal difference error:

$$\delta_t = R_t + \gamma \hat{v}(S_{t+1}; a_{t+1}) - \hat{v}(S_t; a_t) \tag{5}$$

This error is used to update the parameters of both the critic:

$$w \leftarrow w + \alpha_w \cdot \delta_t \cdot \nabla_w \hat{v}(S_t; a) \tag{6}$$

and the actor:

$$\theta \leftarrow \theta + \alpha_\theta \cdot \delta_t \cdot \nabla_\theta \log \pi(A_t|S_t; \theta) \tag{7}$$

This approach supports online learning, with updates occurring at each time step.



Fig. 4. Mean episode reward vs training steps for TD(0) Actor-Critic.

From the Figure 4, the agent exhibits substantial variability throughout training, with some reward collapses followed by quick recoveries. Nevertheless, the overall trend indicates learning progress, with mean reward peaks above 100 and occasional spikes near 400. This highlights the trade-off in TD methods between faster feedback and potential instability during updates.

B. Monte Carlo Actor-Critic

In the Monte Carlo Actor-Critic approach, the critic is updated using the full return computed from complete episodes.

This update yields an unbiased estimate of the return but delays learning until the end of the episode.

The actor is updated using the same policy gradient expression.



Fig. 5. Mean episode reward vs training steps for Monte Carlo Actor-Critic.

The implementation uses a weighted multi-objective loss function: $\mathcal{L}total = 1.0 \cdot \mathcal{L}actor + 0.25 \cdot \mathcal{L}critic + 0.02 \cdot \mathcal{L}entropy$. The dominant actor weight (1.0) prioritizes policy optimization, while the reduced critic weight (0.25) prevents value function overfitting. The small entropy weight (0.02) encourages exploration by penalizing overly deterministic policies without overwhelming the primary learning objectives. The Monte Carlo Actor-Critic method demonstrates a smoother and more consistent learning curve, as shown in Figure 5. These results show that, despite the delay in updates, Monte Carlo Actor-Critic can achieve high final performance with greater training stability.

Overall, both implementations demonstrate effective learning. The TD(0) variant offers faster updates but suffers from higher reward variance, while the Monte Carlo version achieves higher final rewards with smoother progression, albeit with delayed feedback and slightly higher computational costs per episode.

V. REINFORCE VS ACTOR-CRITIC

Compared to REINFORCE, the Monte Carlo Actor-Critic algorithm exhibits significantly improved learning stability and faster convergence. While REINFORCE suffers from high variance due to its reliance on full-episode returns, Actor-Critic integrates a value function that provides a more stable learning signal through the estimated advantage. As shown in the learning curves, REINFORCE oscillates widely and progresses slowly, whereas Monte Carlo Actor-Critic achieves higher and more consistent rewards with smoother progression, thanks to the critic's variance-reducing role.

VI. IMPLEMENTATION OF PPO ON CUSTOMHOPPER

This section presents the implementation of the Proximal Policy Optimization (PPO) algorithm using the `Stable-Baselines3` (SB3) library to train an agent on the `CustomHopper` environment. PPO was selected due to its proven stability and effectiveness in continuous control tasks, particularly in locomotion environments like Hopper.

PPO is a policy gradient method that addresses the challenge of determining appropriate step sizes in policy updates. Unlike traditional policy gradient methods (REINFORCE or Actor-Critic) that can suffer from large policy updates leading to performance collapse, PPO introduces a clipped surrogate objective that constrains policy updates within a trust region. This approach ensures more stable training while maintaining sample efficiency.

The key advantages of PPO for the Hopper environment include:

- **Stability**: The clipping mechanism prevents destructive policy updates.
- **Sample Efficiency**: More efficient use of collected experience compared to vanilla policy gradient methods.
- **Robustness**: Less sensitive to hyperparameter choices.

A. Implementation Architecture

1) Environment Setup and Normalization: The implementation utilizes a sophisticated environment configuration that includes several critical components:

```
train_env = DummyVecEnv([lambda: Monitor(gym.make(
    config.env_id))])
train_env = VecNormalize(train_env, norm_obs=True,
    norm_reward=True)
```

a) Environment Wrapping Strategy:

- **Monitor**: Tracks episode statistics including rewards, lengths, and other metrics for analysis.
- **DummyVecEnv**: Provides a vectorized environment interface, enabling batch processing of environment interactions.
- **VecNormalize**: Applies normalization to both observations and rewards, which is crucial for stable training in continuous control tasks.

This normalization is crucial for achieving more stable and faster convergence, especially in high-variance environments like Hopper.

b) Experiment Tracking: `Weights & Biases (WandB)` was used for experiment tracking, visualization, and hyperparameter logging. This integration provided a powerful way to monitor training metrics and to facilitate reproducibility.

2) Hyperparameters and Model Configuration: The PPO model was instantiated with the following key hyperparameters:

- **Policy architecture**: `MlpPolicy` – a Multi-Layer Perceptron suitable for continuous control.
- **Number of steps per rollout** (`n_steps`): 4096.
- **Batch size**: 256.
- **GAE lambda** (`gae_lambda`): 0.95.
- **Discount factor** (`gamma`): 0.99.
- **Number of epochs per update** (`n_epochs`): 10.
- **Entropy coefficient** (`ent_coef`): 0.01.
- **Learning rate**: $1 \cdot 10^{-4}$.
- **Clipping range** (`clip_range`): 0.2.
- **Total training timesteps**: 1,000,000.

These parameters were selected based on the SB3 documentation and common PPO practices. The entropy coefficient helps to encourage exploration, and a relatively small learning rate was chosen to ensure stable learning. The relatively large rollout length (4096 steps) allows for sufficient experience collection in the dynamic Hopper environment.

3) Callbacks and Evaluation: During training, two callbacks were used:

- **WandbCallback**: Automatically logs metrics such as episode rewards, losses, gradients, and periodically saves model checkpoints to the `wandb` directory.
- **EvalCallback**: Periodically evaluates the policy every 10,000 steps on a separate evaluation environment (normalized only in observation). It saves the best-performing model based on mean reward. Reward normalization was disabled during evaluation to obtain true reward values.

B. Results Analysis and Performance Evaluation

The PPO agent demonstrated excellent learning performance on the `CustomHopper` environment, as evidenced by the evaluation metrics collected during training (see Fig. 6).

The evaluation results reveal a highly successful training process with distinct phases:

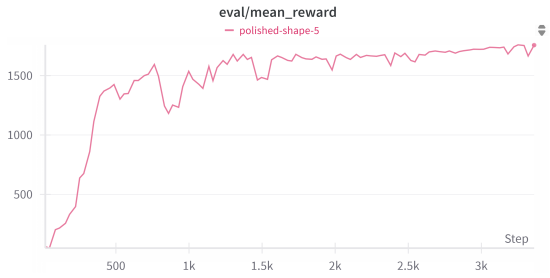


Fig. 6. Evaluation rewards over time during PPO training on the CustomHopper environment. (The x-axis in the graph is scaled proportionally to 1,000,000)

1) **Rapid Initial Learning (Steps 0–140,000)**: The agent exhibits extremely fast initial improvement, progressing from approximately 100 to 1400 mean reward within the first 140,000 evaluation steps. This steep learning curve indicates that the PPO algorithm quickly discovered fundamental locomotion strategies in the Hopper environment.
2) **Exploration and Refinement Phase (Steps 140,000–450,000)**: Following the initial rapid improvement, the agent enters a phase characterized by exploration and policy refinement. The mean reward oscillates between 1200–1700, showing some volatility as the agent explores different locomotion strategies.
3) **Stabilization and Convergence (Steps 450,000–1,000,000)**: The final phase demonstrates excellent convergence characteristics, with the mean reward stabilizing around 1700–1800. The reduced variance in this phase indicates that the agent has converged to a stable, high-performance policy.

VII. Sim-to-Real Transfer Analysis and Domain Adaptation

In real-world robotics applications, direct training on the real system is often infeasible due to safety risks, limited access, or high operational costs. For this reason, reinforcement learning models are commonly trained in simulation and then transferred to the real world—a process known as *sim-to-real transfer*.

This section presents a comprehensive study of domain adaptation through a controlled sim-to-sim experimental setup that simulates the reality gap encountered when deploying policies trained in simulation to real-world environments.

A. Domain Gap Simulation

To replicate the sim-to-real scenario in a controlled manner, two distinct `CustomHopper` environments were created:

- **Source Domain:** Features an imprecise torso mass with a **-30% shift** from nominal physical parameters.
- **Target Domain:** Maintains the original, accurate physical parameters.

The experimental framework evaluates three critical transfer configurations to provide insight into the domain adaptation challenge:

B. Transfer Configuration Analysis

1) **Source→Source (Baseline Performance)**
   - *Purpose:* Establish the performance ceiling for policies trained and tested in the same domain.
   - *Expected Outcome:* Maximum achievable performance given the source environment dynamics.
   - *Significance:* Provides a baseline to measure transfer degradation.
2) **Source→Target (Reality Gap Impact)**
   - *Purpose:* Quantify the performance penalty imposed by domain mismatch.
   - *Expected Outcome:* Degraded performance due to dynamics mismatch between training and deployment.
   - *Significance:* Represents the core sim-to-real transfer challenge and its lower bound.
3) **Target→Target (Upper Bound Performance)**
   - *Purpose:* Establish the theoretical maximum performance achievable in the target domain.
   - *Expected Outcome:* Optimal performance representing perfect domain knowledge.
   - *Significance:* Provides an upper bound for transfer learning effectiveness.

C. Hyperparameter Optimization Framework

Given the critical role of hyperparameter selection in transfer robustness, a Bayesian optimization approach was implemented using `Weights & Biases` sweep functionality.

Optimization Strategy: The hyperparameter search (Table I) employed Bayesian optimization, which constructs a probabilistic model of the objective function to guide exploration. This strategy is particularly efficient for computationally expensive evaluations such as RL training, outperforming grid and random search in both speed and sample efficiency.

D. Experimental Results and Performance Analysis

Following hyperparameter optimization, the best-performing configurations were evaluated across all three transfer scenarios using 50 test episodes to ensure statistical significance (Table I).

The **Source→Target** configuration results highlight the degradation due to domain mismatch. The policy trained on source dynamics performs significantly worse when deployed in the target environment.

The gap between the **Source→Target** and **Target→Target** configurations reflects a **performance loss of 8.03%**, highlighting the impact of the sim-to-real reality gap.

TABLE I
COMPARISON OF PPO HYPERPARAMETERS AND MEAN REWARD ACROSS TRANSFER SETTINGS

Hyperparameter	Source→Source	Source→Target	Target→Target
n_steps	8192	8192	8192
batch size	512	64	512
gae_lambda	0.935	0.988	0.979
gamma	0.995	0.990	0.991
n_epochs	15	20	20
ent_coef	0.00246	0.03453	0.00795
learning rate	9.83e-4	1.02e-5	3.51e-4
clip_range	0.2232	0.2947	–
Mean reward	1703.21	1625.02	1766.93
Std Deviation	24.00	16.36	23.66

E. Causes of Performance Degradation

The observed degradation in the Source→Target transfer stems from several domain adaptation challenges:

- **Dynamics Mismatch:** The 30% torso mass reduction alters the robot's center of mass, moment of inertia, and overall dynamics, requiring distinct control strategies.
- **Policy Specialization:** Policies trained in the source domain become specialized for those dynamics, making them less effective in the target setting.
- **Exploration Limitations:** Training on source dynamics may lead to under-exploration of critical behaviors relevant in the target domain.
- **Feature Representation:** Internal policy representations may capture domain-specific patterns that do not generalize to new dynamics.

F. Practical Constraints of Target Training

In practical applications, direct training in the target (real-world) environment is rarely feasible due to:

- **Unknown Parameters:** Real-world physical parameters are often uncertain or difficult to model accurately.
- **Safety Risks:** Exploration on physical hardware can be dangerous or damaging.
- **Operational Costs:** Real robot experiments are time-consuming and expensive.
- **Sample Efficiency:** Real-world training requires significantly more wall-clock time.

G. Conclusion of Transfer Study

Despite the inherent performance drop caused by domain mismatch, the results demonstrate that:

- Careful algorithmic design,
- Robust training methodologies,
- And systematic hyperparameter optimization

can substantially mitigate sim-to-real degradation. The **91.97% performance retention** in the Source→Target scenario supports the feasibility of simulation-based training for real-world robotic applications, establishing a strong foundation for future transfer learning research and deployment.

VIII. Uniform Domain Randomization Implementation and Analysis

A. Theoretical Foundation and Motivation

Uniform Domain Randomization (UDR) is a foundational technique aimed at improving policy robustness by exposing the agent to a wide range of environmental conditions during training. The core hypothesis behind UDR is that policies trained across a distribution of environments will develop more generalizable behaviors capable of handling domain shifts at deployment time.

UDR addresses the sim-to-real transfer challenge by shifting the training paradigm from single-domain optimization to multi-domain robustness learning. Rather than learning an optimal behavior for a specific set of dynamics, the agent learns strategies that perform adequately across a range of physical configurations. This promotes robustness through exposure to variability.

In our implementation, UDR was applied by randomizing the three lower-body link masses—`thigh`, `leg`, and `foot`—while keeping the `torso` mass fixed at 30% below its target value. This setup preserves a constant domain gap (via the torso) while injecting episode-level variability through randomized dynamics.

B. Randomization Strategy and Configurations

A set of six configurations was designed to systematically explore how different mass ranges impact policy robustness. Each configuration tested a specific hypothesis regarding the shape, width, or shift of the randomization distribution.

These configurations are summarized in Table II.

TABLE II
Domain Randomization Configurations for Limb Masses

Configuration	Thigh Range	Leg Range	Foot Range
A (Narrow)	(0.8, 1.2)	(0.8, 1.2)	(0.8, 1.2)
B (Medium)	(0.5, 1.5)	(0.5, 1.5)	(0.5, 1.5)
C (Wide)	(0.3, 1.7)	(0.3, 1.7)	(0.3, 1.7)
D (Heavy)	(1.0, 2.0)	(1.0, 2.0)	(1.0, 2.0)
E (Light)	(0.2, 1.0)	(0.2, 1.0)	(0.2, 1.0)
F (Irregular)	(0.8, 1.5)	(0.4, 1.2)	(0.6, 1.4)

C. Performance Analysis

TABLE III
Mean Episode Reward (± std) for Each Randomization Configuration in Source→Source and Source→Target

Config	Source→Source	Source→Target
A	1184.7 ± 172.3	1032.7 ± 198.0
B	1236.6 ± 98.7	1006.2 ± 104.8
C	1254.1 ± 210.2	1107.7 ± 143.0
D	984.4 ± 68.2	845.5 ± 143.6
E	1486.6 ± 66.1	1520.0 ± 118.8
F	1480.9 ± 81.7	1615.7 ± 90.9

From Table III, experiments **E** and **F**—featuring lighter mass distributions—achieved superior performance. This suggests that UDR effectiveness depends significantly on the alignment between the randomized training distribution and the true target dynamics.

- **Conservative randomization (A):** Led to poor performance and high variance due to limited variability.
- **Aggressive randomization (C):** Maintained average performance, but increased learning variance and instability.

UDR proved to be a powerful tool for increasing the transferability of policies across domain shifts. By training over a distribution of dynamics, the agent learned behaviors that generalized beyond a single environment instance.

The best UDR configuration achieved a **mean reward of 1615.7**, which is remarkably close to the **source→target baseline performance of 1625.02**. This confirms that UDR can serve as a viable method to approximate sim-to-real conditions in simulation.

D. Limitations of UDR

Despite its benefits, UDR also comes with several notable drawbacks:

1) **Manual tuning:** Choosing appropriate parameter distributions is time-consuming and highly domain-specific.
2) **Over-randomization risk:** Excessive variability (as in configuration C) can delay convergence or destabilize training.
3) **Lack of optimality:** UDR promotes robustness but does not guarantee optimal performance in any specific domain.
4) **Computational overhead:** Training is slower due to environment re-initialization, high variance, and increased episode count requirements.

E. Conclusion

In summary, UDR represents a valuable step toward achieving generalization in sim-to-real transfer. Its effectiveness depends heavily on thoughtful design of parameter distributions and the nature of the target environment. When applied with care, UDR can significantly enhance policy robustness and prepare agents for deployment in variable real-world scenarios.

IX. Project Extension

As an extension to the main experiments conducted in the Hopper environment, the generalization capability of the proposed reinforcement learning pipeline was investigated on a different robotic control task. Specifically, the same methodology was applied to the `Reacher` environment from OpenAI Gym, which presents a distinct control challenge and serves to assess the adaptability of the implementation.

A. Reacher Environment

The Reacher environment simulates a two-link robotic arm operating in a 2D plane. The task requires the arm to reach a randomly placed target as quickly and accurately as possible. Unlike the Hopper task, which emphasizes dynamic balance and locomotion, Reacher focuses on precise motor coordination and control in a stationary setting.

The observation space consists of 11 continuous features, including joint angles, angular velocities, and the vector from the fingertip to the target. The action space is a two-dimensional continuous vector representing the torques applied to each joint.

This environment presents several notable differences compared to Hopper:

- It is fully actuated and stable, with no risk of falling.
- The objective is positional rather than dynamic: reaching a static goal.
- Episodes are typically shorter and less stochastic.
- Unlike the Hopper environment, where the reward is generally positive and increases with forward progress, the Reacher environment is characterized by negative rewards. In Reacher, the agent is penalized at each timestep based on the distance between the end-effector and the target position, as well as for the magnitude of the actions taken. As a result, the total reward is typically negative, and performance is considered better when the cumulative reward is closer to zero.

B. Extension to Reacher Environment

For the *Reacher* environment, the same domain gap simulation applied to *Hopper* was adopted, generating two distinct domains:

- **Source Domain**: Modified dynamics obtained by altering body segment masses as follows: mass of segment 1 reduced by 25%, mass of segment 2 increased by 30%.
- **Target Domain**: Default mass parameters, with no modifications applied.

The PPO implementation for *Reacher* used the following hyperparameter configuration:

- **Policy**: MLP
- **Total timesteps**: 500,000
- **n_steps**: 2048
- **Batch size**: 64
- **GAE** $\lambda = 0.95$
- **Discount factor** $\gamma = 0.99$
- **Entropy coefficient**: 0.01
- **Learning rate**: 3×10^{-4}
- **Clip range**: 0.2
- **Epochs**: 10

Observation and reward normalization were applied to the training environment to enhance learning stability. The evaluation environment employed only observation normalization to ensure consistent state scaling without affecting reward interpretation.

Comprehensive monitoring was implemented through *Weights & Biases* integration for experiment tracking and visualization. Additionally, periodic evaluations were performed every 5,000 steps using deterministic policy execution (**Figure 2**).

The training process exhibited the following learning dynamics (see Fig. 7):



Fig. 7. Evaluation rewards over time during PPO training on the Custom-Reacher environment. (The x-axis in the graph is scaled proportionally to 1,000,000)

- **Initial Phase (0 – 35,000 steps)**: Rapid improvement in mean reward from approximately -26 to -8, indicating quick adaptation to basic reaching mechanics.
- **Stabilization Phase (35,000 – 70,000 steps)**: Convergence to approximately -5 mean reward with reduced variance, suggesting policy stabilization.
- **Plateau Phase (70,000+ steps)**: Performance stabilized between -3 and -5 mean reward, with occasional fluctuations, reflecting mature policy behavior.

Upon completing training, the policy was evaluated over 50 episodes in the source domain, achieving a mean reward of -4.25 with a standard deviation of 2.77. Notably, 64% of the episodes obtained a reward better than -6.0, indicating reliable task execution across diverse initial conditions and target positions.

C. Training and Testing

Following the baseline PPO implementation, extensive hyperparameter optimization was performed using Bayesian optimization to identify optimal configurations for both source and target domains (Table IV).

TABLE IV
COMPARISON OF PPO HYPERPARAMETERS AND MEAN REWARD ACROSS TRANSFER SETTINGS

Hyperparameter	**Source→Source**	**Source→Target**	**Target→Target**
n_steps	4096	4096	1024
batch size	32	32	64
gae_lambda	0.875	0.875	0.967
gamma	0.991	0.991	0.993
n_epochs	15	15	8
ent_coef	0.0641	0.0641	0.0499
learning rate	4.67e-4	4.67e-4	5.21e-4
clip_range	0.190	0.190	0.128
Mean reward	-3.632	-4.146	-3.430
Std Deviation	1.680	1.696	1.327

All configurations were trained for a fixed budget of 500,000 timesteps to ensure fair comparisons.

The top-performing models, based on source domain reward during training, were evaluated over 50 episodes, yielding the results reported in (Table IV). Key observations include:

- **Transfer Gap**: The performance difference between source $\rightarrow$ source (-3.63) and source $\rightarrow$ target (-4.15)

highlights a measurable but manageable domain shift, corresponding to approximately 14% performance degradation.

- **Target Advantage**: Training directly in the target domain achieved slightly superior performance (-3.43) compared to the source domain baseline (-4.14), suggesting that mass alterations in the source domain introduce additional complexity.
- **Variance Consistency**: Standard deviations remained relatively stable across domains (1.33 to 1.70), indicating consistent policy behavior irrespective of domain configuration.

The identified optimal configurations serve as reliable baselines for subsequent Uniform Domain Randomization (UDR) experiments.

D. Uniform Domain Randomization

To enhance the generalization capability of the PPO agent under dynamics mismatches between source and target domains, UDR was implemented by randomizing the masses of the forearm (segment 3) and fingertip (segment 4), while maintaining fixed mass modifications for segments 1 and 2, consistent with the source domain (Table V).

TABLE V
DOMAIN RANDOMIZATION RANGES FOR FOREARM AND FINGERTIP MASSES

Config	forearm Range	fingertip Range
A	(0.5, 1.5)	(0.5, 1.5)
B	(0.6, 1.8)	(0.6, 1.2)
C	(1.0, 1.6)	(1.0, 1.7)
D	(0.5, 1.0)	(0.5, 1.0)
E	(0.3, 0.8)	(0.3, 1.0)
F	(0.3, 1.8)	(0.3, 1.8)

All agents were trained for the same number of steps and evaluated over 50 episodes in both source and target domains (Table VI).

TABLE VI
MEAN EPISODE LOSS ($\pm$ STD) FOR EACH RANDOMIZATION CONFIGURATION IN S$\rightarrow$S AND S$\rightarrow$T

Config	Source$\rightarrow$Source	Source$\rightarrow$Target
A	-4.422 ± 1.81	-3.501 ± 1.31
B	-4.160 ± 1.35	-3.942 ± 1.97
C	-3.923 ± 2.33	-4.031 ± 1.69
D	-3.732 ± 1.25	-3.633 ± 1.39
E	-3.607 ± 1.48	-3.349 ± 1.42
F	-3.796 ± 1.35	-4.027 ± 1.06

The best transfer performance was achieved by **Configuration E**, which exhibited minimal reward degradation between source and target domains.

However, **Configuration A** demonstrated a favorable balance between overall training performance and transferability, consistently yielding high rewards in both environments.

Compared to the naive PPO transfer from source to target (mean reward = -4.15), most UDR agents achieved superior or comparable performance, with particularly robust results in configurations A and E.

Overall, UDR led to a significant 16% improvement in target domain performance relative to standard source domain training, validating the effectiveness of domain randomization for sim-to-sim transfer in the *Reacher* environment.

X. CONCLUSION

In this project, reinforcement learning techniques were investigated for robotic control in simulated environments, with a particular emphasis on the sim-to-real transfer problem. The implementation of REINFORCE, Actor-Critic (including both TD(0) and Monte Carlo variants), and PPO allowed for an analysis of the trade-offs between sample efficiency, stability, and final performance.

Experimental results showed that Actor-Critic methods, particularly the Monte Carlo variant, outperformed REINFORCE in terms of convergence speed and learning stability. PPO, implemented using Stable-Baselines3, achieved state-of-the-art performance, demonstrating both rapid learning and high robustness in the Hopper environment.

To simulate the sim-to-real transfer challenge, a domain shift was introduced by modifying the robot's torso mass, and policy transfer performance was evaluated across varying domain configurations. Uniform Domain Randomization (UDR) proved effective in reducing the transfer gap by increasing policy robustness through training variability.

The pipeline was further extended to the Reacher environment to assess the generality of the approach. The results confirmed the adaptability of the proposed methods to tasks with different dynamics, and demonstrated the effectiveness of UDR even in low-dimensional, precision-based control problems.

Overall, the findings emphasize the importance of combining advanced policy optimization techniques with principled regularization strategies, such as domain randomization, to achieve robust policy transfer across domains. This work provides a solid foundation for future research on sim-to-real transfer in robotics, including extensions to more complex scenarios and real-world applications.

REFERENCES

[1] Farama Foundation, "MuJoCo environments," in *Gymnasium Documentation*, 2024. Available: gymnasium.farama.org/environments/mujoco/

[2] L. Weng, "Policy Gradient Algorithms," Lil'Log (Lilian Weng's blog), Apr. 8, 2018.

[3] P. Kormushev, S. Calinon, and D. G. Caldwell, "Reinforcement learning in robotics: Applications and real-world challenges," 2013.

[4] S. Höfer *et al.*, "Perspectives on sim2real transfer for robotics: A summary of the R:SS 2020 workshop," 2020.

[5] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," arXiv preprint arXiv:1703.06907, 2017.

[6] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint arXiv:1707.06347, 2017.

[8] F. Muratore *et al.*, "Robot learning from randomized simulations: A review," *Frontiers in Robotics and AI*, vol. 9, 2022.